TPI BLANC

Infrastructure virtualisée Proxmox avec services Docker interconnectés

Candidat : Antoine BERGER
Chef de projet : Maeva LONDERO
EPFL – ENAC-IT
Mars 2026

Table des matières

1. Analyse préliminaire 3
1.1 Introduction 3
1.2 Objectifs 3
1.3 Planification initiale 4
2. 2. Analyse / Conception 5
2.1 Concept 5
2.2 Stratégie de test 5
2.3 Risques techniques 6
2.4 Planification 6
2.5 Dossier de conception 6
3. 3. Réalisation 8
3.1 Dossier de réalisation 8
3.2 Description des tests effectués 13
3.3 Erreurs restantes 15
3.4 Liste des documents fournis 16
4. 4. Conclusions 17
5. 5. Annexes 18
5.1 Résumé du rapport du TPI 18
5.2 Sources – Bibliographie 18
5.3 Journal de travail 20
5.4 Manuel d'installation 20
5.5 Glossaire 20
5.6 Archives du projet 20

1. Analyse préliminaire

1.1 Introduction

Ce projet s'inscrit dans le cadre du TPI blanc (travail pratique individuel d'entraînement), réalisé sur 3 jours à l'EPFL au sein de l'équipe ENAC-IT. Il vise à mettre en place une infrastructure virtualisée complète basée sur l'hyperviseur Proxmox VE, hébergeant plusieurs conteneurs LXC faisant tourner des services Docker interconnectés.

L'objectif fonctionnel est de déployer une stack applicative composée d'un reverse proxy Nginx, d'une application de gestion de tâches Kanboard et d'une base de données PostgreSQL, le tout réparti sur plusieurs conteneurs LXC et communiquant à travers un réseau interne configuré au niveau de Proxmox.

Ce projet permet de mobiliser des compétences en virtualisation, administration Linux, conteneurisation Docker, configuration réseau et documentation technique — compétences centrales de l'orientation Exploitation et Infrastructure du CFC d'informaticien.

La méthode de gestion de projet appliquée est une **approche séquentielle (cascade)** adaptée à la durée de 3 jours : analyse et planification initiale, conception, réalisation, tests, puis documentation. Cette méthode est visible dans le découpage du Gantt et dans la structure du présent rapport.

1.2 Objectifs

Les objectifs du projet, tels que définis dans le cahier des charges, sont les suivants :

Installation et configuration de Proxmox VE — Installer l'hyperviseur Proxmox sur la VM fournie par le service xaas de l'EPFL. Configurer le réseau de base et le stockage.

Déploiement des conteneurs LXC et communication inter-conteneurs — Créer les conteneurs LXC nécessaires, installer Docker sur chacun, et configurer le réseau pour qu'ils puissent communiquer entre eux de manière sécurisée. La base de données ne doit être accessible que depuis le conteneur applicatif.

Stack applicative fonctionnelle — Déployer via docker-compose les services suivants : Nginx (reverse proxy), Kanboard (application web), PostgreSQL (base de données). Le flux doit fonctionner de bout en bout sans intervention manuelle.

Persistance et résilience — Les données doivent persister après redémarrage des containers. Au moins un snapshot Proxmox doit être créé et la procédure de restauration documentée.

Documentation professionnelle — Fournir une documentation complète avec schéma d'architecture réseau, procédures d'installation, fichiers de configuration commentés et journal de travail quotidien.

Note : le cahier des charges prévoyait initialement des VMs. L'utilisation de conteneurs LXC est un ajustement technique justifié en section 2.3 (Risques techniques).

1.3 Planification initiale

La planification initiale est présentée sous forme de diagramme de Gantt (cf. fichier PlanificationInitiale_Blanc_AB.gantt). Le projet se déroule du lundi 23 mars au mercredi 25, et l'après-midi du vendredi 27 mars 2026, selon les horaires suivants :

Lundi: 10h-12h / 12h30 – 16h

Mardi: 8h30-12h / 12h30 – 17h

Mercredi: 10h-12h / 12h30-17h

Vendredi: 12h30-16h30

Total = 24h de travail

La répartition indicative du temps est la suivante : analyse et planification (~10%, soit 2.5h), implémentation (~60%, soit 14.5h), tests et validation (~5%, soit 1h), documentation (~25%, soit 6h).

2. Analyse / Conception

2.1 Concept

L'architecture cible repose sur trois couches :

Couche 1 — Hyperviseur : Proxmox VE installé en bare-metal sur la VM xaas de l'EPFL (16 Go RAM, 8 CPU, IP EPFL 10.95.48.44). L'ISO a été ajoutée au catalogue xaas avec l'aide d'un collègue disposant des droits nécessaires. Proxmox fournit la gestion des conteneurs LXC, la gestion réseau (bridges virtuels) et les snapshots. La VM xaas ne supportant pas KVM (virtualisation imbriquée désactivée), les instances invitées sont des conteneurs LXC plutôt que des VMs classiques.

Couche 2 — Conteneurs LXC : 3 conteneurs LXC basés sur le template Debian 12 (debian-12-standard_12.12-1), créés dans Proxmox en mode non-privilégié avec nesting activé. La communication inter-conteneurs est assurée par un bridge interne Proxmox (vmbr1) avec des IPs statiques sur le subnet 10.10.10.0/24. L'Hôte Proxmox (10.10.10.1) fait office de passerelle et assure le NAT pour l'accès internet des conteneurs.

Couche 3 — Services Docker :

- **CT1 (10.10.10.10) :** Nginx en reverse proxy — point d'entrée HTTP, redirige vers CT2
- **CT2 (10.10.10.20) :** Kanboard — application web de gestion de tâches
- **CT3 (10.10.10.30) :** PostgreSQL — base de données, accessible uniquement depuis CT2

Flux applicatif : Client → Port forward Proxmox (:80) → Nginx (CT1) → Kanboard (CT2) → PostgreSQL (CT3)

Un schéma d'architecture réseau détaillé a été réalisé avec draw.io et est inclus dans le dossier du projet.

2.2 Stratégie de test

Les tests sont organisés par couche, de bas en haut :

Couche	Test	Méthode	Résultat attendu
Conteneurs LXC	Démarrage correct des 3 CT	pct list, pct enter	Status: running
Réseau	Connectivité inter-CT sur vmbr1	ping entre CT	0% packet loss
Réseau	Isolation DB (CT1 ne peut pas joindre CT3:5432)	iptables + test connexion	Connexion refusée
Réseau	Accès internet des CT (NAT)	ping 8.8.8.8	0% packet loss
Services Docker	Containers applicatifs en fonctionnement	docker compose ps	Status: Up / healthy
Bout en bout	Accès Kanboard via reverse proxy	curl -I http://IP	HTTP 302 → /login
Bout en bout	Création d'une tâche dans Kanboard	Interface web	Tâche visible + persistante
Persistance	Données après redémarrage Docker	docker compose restart + vérif	Données présentes

Snapshot	Restauration snapshot Proxmox	pct snapshot / rollback	Services fonctionnels

Les preuves de test seront fournies sous forme de captures d'écran et de logs CLI.

2.3 Risques techniques

Risque	Impact	Statut	Mitigation
Connectivité à la VM hôte (SSH, réseau EPFL)	Bloquant	**Résolu**	Clés SSH ed25519 générées. Connexion testée et fonctionnelle.
Proxmox non supporté sur Ubuntu. (Installation non viable pour projet.)	Bloquant	**Résolu**	Utilisation de l'iso de Proxmox pour installation en bare-metal sur la VM Xaas (réinstallation requise)
ISO Proxmox indisponible dans le catalogue xaas	Bloquant	**Résolu**	ISO ajoutée avec l'aide d'un collègue (Nicolas Dubois) disposant des droits.
KVM non supporté sur la VM xaas (virtualisation imbriquée désactivée)	Bloquant	**Résolu**	Utilisation de conteneurs LXC à la place des VMs KVM. Résultat fonctionnel équivalent : 3 instances isolées avec Docker. Déviation par rapport au CDC signalée.
Docker dans LXC — configuration spécifique requise (nesting)	Moyen	**Résolu**	Option nesting=1 activée à la création (--features nesting=1). Docker 29.3.0 installé et fonctionnel sur les 3 CT.
Complexité réseau inter-conteneurs (bridges, firewall, routage)	Moyen	**Résolu**	Bridge vmbr1 créé avec IPs statiques, NAT fonctionnel. Port forwarding et restriction d'accès DB restants.
Thin pool Proxmox — avertissement de dépassement capacité	Faible	**Surveillé**	Thin provisioning n'alloue que l'espace réellement utilisé. Surveiller l'utilisation disque. (**51.9% in use**)
Temps limité (24h) pour infrastructure complète + documentation	Élevé	**Résolu**	Prioriser le stack fonctionnel de bout en bout, puis la documentation et les finitions.

2.4 Planification

La planification définitive est maintenue dans le fichier PlanificationInitiale_Blanc_AB.gantt.

Pour une question de lisibilité, le planning est affiché sous le point "Archives du projet". Elle sera comparée à l'avancement réel en fin de projet pour évaluer les écarts.

2.5 Dossier de conception

Matériel / Infrastructure :

- VM hôte : fournie par le service xaas de l'EPFL — 8 CPU, 16 Go RAM, 40 Go + 150 Go SSD, IP 10.95.48.44
- Accès : interface web Proxmox (port 8006) + shell intégré

Systèmes d'exploitation :

- VM hôte : Proxmox VE 9.1.1, installé directement depuis l'ISO
- Conteneurs LXC : Debian 12 Bookworm (template standard Proxmox debian-12-standard_12.12-1)

Logiciels de réalisation :

- Hyperviseur : Proxmox VE (interface web port 8006)
- Conteneurisation : Docker Engine 29.3.0 + docker-compose (plugin intégré)
- Reverse proxy : Nginx stable (image Docker officielle nginx:stable)
- Application : Kanboard latest (image Docker officielle kanboard/kanboard:latest)
- Base de données : PostgreSQL 16 (image Docker officielle postgres:16)
- Éditeur de configuration : nano (CLI)
- Schémas réseau : draw.io

Logiciels d'utilisation :

- Navigateur web Brave (interface Proxmox + Kanboard)

Gestion de version / livrables :

- Documentation dans un fichier readme.md
- Fichiers de configuration versionnés (docker-compose.yml, nginx.conf, etc.)

3. Réalisation

3.1 Dossier de réalisation

Configuration réseau Proxmox

Le bridge `vmbr0` (réseau EPFL, 10.95.48.44/24) existait déjà à l'installation de Proxmox. Un second bridge `vmbr1` a été créé manuellement dans `/etc/network/interfaces` avec les paramètres suivants :

- Adresse : 10.10.10.1/24 (l'Hôte Proxmox sert de passerelle)
- NAT activé via iptables (règle MASQUERADE sur vmbr0) pour permettre l'accès internet des conteneurs
- IP forwarding activé (`echo 1 > /proc/sys/net/ipv4/ip_forward`)

Création des conteneurs LXC

Les 3 conteneurs ont été créés via `pct create` à partir du template `debian-12-standard_12.12-1_amd64.tar.zst` avec les options : mode non-privilégié (`--unprivileged 1`) et nesting activé (`--features nesting=1`). Chaque conteneur est connecté à vmbr1 avec une IP statique.

La fonctionnalité `nesting` permet de déployer des conteneurs Docker à l'intérieur des conteneurs LXC.

CT	ID	Hostname	IP	RAM	Cores	Disque	Rôle
CT1	101	ct1-nginx	10.10.10.10	512 Mo	1	8 Go	Nginx reverse proxy
CT2	102	ct2-kanboard	10.10.10.20	1024 Mo	2	10 Go	Kanboard (app web)
CT3	103	ct3-postgres	10.10.10.30	512 Mo	1	10 Go	PostgreSQL (DB)

Installation de Docker

Docker Engine 29.3.0 a été installé sur les 3 conteneurs via le dépôt officiel Docker pour Debian. La procédure comprend l'ajout de la clé GPG, du dépôt APT, puis l'installation des paquets `docker-ce`, `docker-ce-cli`, `containerd.io` et `docker-compose-plugin`.

Déploiement des services

Chaque service est déployé via un fichier `docker-compose.yml` dans `/opt/<service>/` sur le conteneur correspondant :

- **CT3 — PostgreSQL** (`/opt/postgres/docker-compose.yml`) : image `postgres:16`, base `EXEMPLE_DB`, volume persistant `pgdata` pour les données.
- **CT2 — Kanboard** (`/opt/kanboard/docker-compose.yml`) : image `kanboard/kanboard:latest`, connexion à la DB via `DATABASE_URL` pointant sur 10.10.10.30:5432. Volumes persistants pour les données applicatives et les plugins.

- **CT1 — Nginx** (`/opt/nginx/docker-compose.yml` + `nginx.conf`) : image `nginx:stable`, reverse proxy vers 10.10.10.20:8080. Le fichier `nginx.conf` configure le `proxy_pass` et les headers de forwarding.

Test bout en bout

Un `curl -I http://localhost` depuis CT1 et un `curl -I http://10.10.10.10` depuis l'hôte retournent tous deux un `HTTP/1.1 302 Found` avec redirection vers `/login`, confirmant le fonctionnement de la chaîne complète : Nginx → Kanboard → PostgreSQL.

Port forwarding (accès depuis le réseau EPFL)

Pour rendre Kanboard accessible depuis le réseau EPFL, une règle DNAT redirige le trafic HTTP entrant sur l'IP EPFL (10.95.48.44:80) vers CT1 (10.10.10.10:80) :

```
iptables -t nat -A PREROUTING -i vmbr0 -p tcp --dport 80 -j DNAT --to-destination 10.10.10.10:80
```

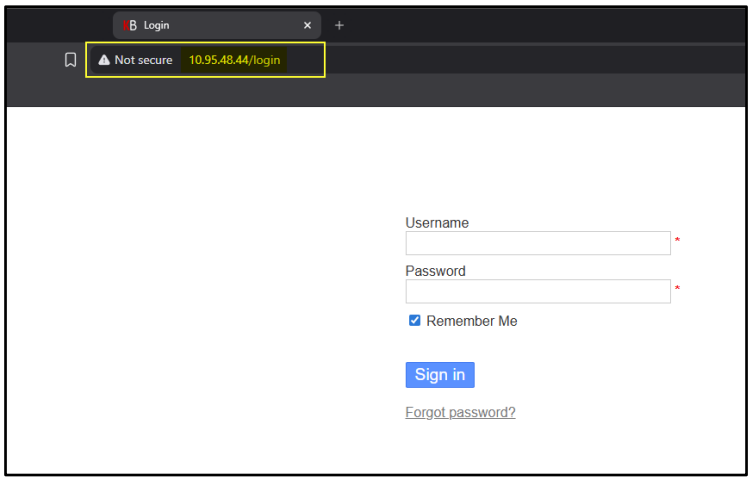
- `-i vmbr0` : uniquement le trafic arrivant depuis le réseau EPFL
- `--dport 80` : port HTTP
- `--to-destination 10.10.10.10:80` : redirige vers Nginx sur CT1

Lorsqu'une règle dans iptables est créée, il est important de créer son inverse, afin qu'en cas de redémarrage du bridge, les règles soient nettoyées. De cette façon nous évitons de créer des doublons qui peuvent causer des erreurs par la suite. (ce principe a été appliqué pour toutes les règles écrites dans iptables)

```
post-down iptables -t nat -D PREROUTING -i vmbr0 -p tcp --dport 80 -j DNAT --to-destination 10.10.10.10:80
```

Vérification : depuis un navigateur sur le réseau EPFL, l'accès à `http://10.95.48.44` affiche la page de login Kanboard.

Capture d'écran :



Page de connexion de l'application Kanboard, accessible sur l'ip de l'Hôte.

Isolation de la base de données

Les conteneurs LXC étant connectés au même bridge (vmbr1), le trafic entre eux est commuté au niveau L2 et ne passe pas par la chaîne FORWARD d'iptables. Pour que les règles de filtrage s'appliquent, le module br_netfilter a été activé sur l'hôte (`modprobe br_netfilter`) avec l'option `net.bridge.bridge-nf-call-iptables = 1`. Sans cette étape, les règles FORWARD n'avaient aucun effet — ce qui a été identifié grâce aux compteurs iptables restés à zéro lors des premiers tests. Une fois cette étape activée, les compteurs démarrent.

Deux règles iptables sur l'hôte Proxmox restreignent l'accès à CT3 (PostgreSQL, port 5432) au seul CT2 (Kanboard) :

1) `iptables -A FORWARD -s 10.10.10.20 -d 10.10.10.30 -p tcp --dport 5432 -j ACCEPT`

2) `iptables -A FORWARD -d 10.10.10.30 -p tcp --dport 5432 -j DROP`

L'ordre est crucial : la règle ACCEPT doit être insérée avant la règle DROP, sinon CT2 serait également bloqué.

Vérification : depuis CT2, une connexion PostgreSQL vers CT3 réussit. Depuis CT1, la même tentative aboutit à un timeout, confirmant l'isolation.

Capture d'écran :

```
root@ct1-nginx:/# psql -h 10.10.10.30 -U kanboard -d kanboard -c "SELECT 1;"
perl: warning: Setting locale failed.
perl: warning: Please check that your locale settings:
        LANGUAGE = (unset),
        LC_ALL = (unset),
        LANG = "en_US.UTF-8"
    are supported and installed on your system.
perl: warning: Falling back to the standard locale ("C").
psql: error: connection to server at "10.10.10.30", port 5432 failed: Connection timed out
        Is the server running on that host and accepting TCP/IP connections?
root@ct1-nginx:/# exit
exit
root@proxmox:/# iptables -L FORWARD -n -v
Chain FORWARD (policy ACCEPT 69991 packets, 687M bytes)
 pkts bytes target     prot opt in     out     source               destination
 2472  375K ACCEPT     tcp  --  vmbr0  vmbr1   0.0.0.0/0            10.10.10.10          tcp dpt:80
  126 10577 ACCEPT     tcp  --  *      *       10.10.10.20          10.10.10.30          tcp dpt:5432
   11   660 DROP       tcp  --  *      *       0.0.0.0/0            10.10.10.30          tcp dpt:5432
root@proxmox:/#
```

Tentatives de requêtes sur la base de données depuis CT1 + affichage des compteurs iptables

Persistance des règles réseau

Les règles iptables (NAT, port forwarding, isolation DB) sont volatiles et perdues au redémarrage. Pour les rendre persistantes, elles ont été ajoutées dans `/etc/network/interfaces` via des directives `post-up` dans le bloc `vmbr1` :

- `post-up iptables -t nat -A PREROUTING -i vmbr0 -p tcp --dport 80 -j DNAT ...` (port forwarding)
- `post-up iptables -A FORWARD -s 10.10.10.20 -d 10.10.10.30 ... -j ACCEPT` (isolation DB)
- `post-up iptables -A FORWARD -d 10.10.10.30 ... -j DROP` (isolation DB)

Ainsi, l'ensemble de la configuration réseau est restaurée automatiquement au démarrage du bridge. Aussi, toutes les configurations concernant le bridge sont au même endroit et permettent une maintenance facilitée par la suite. Le fichier `/etc/network/interfaces` complet est documenté dans le README.

Capture d'écran :

```
auto lo
iface lo inet loopback

iface nic0 inet manual

auto vmbr0
iface vmbr0 inet static
        address 10.95.48.44/24
        gateway 10.95.48.1
        bridge-ports nic0
        bridge-stp off
        bridge-fd 0


auto vmbr1
iface vmbr1 inet static
        address 10.10.10.1/24
        bridge-ports none
        bridge-stp off
        bridge-fd 0
        post-up   echo 1 > /proc/sys/net/ipv4/ip_forward
        post-up   iptables -t nat -A POSTROUTING -s 10.10.10.0/24 -o vmbr0 -j MASQUERADE
        post-down iptables -t nat -D POSTROUTING -s 10.10.10.0/24 -o vmbr0 -j MASQUERADE
        post-up   iptables -t nat -A PREROUTING -i vmbr0 -p tcp --dport 80 -j DNAT --to-destination 10.10.10.10:80
        post-down iptables -t nat -D PREROUTING -i vmbr0 -p tcp --dport 80 -j DNAT --to-destination 10.10.10.10:80
        post-up   iptables -A FORWARD -s 10.10.10.20 -d 10.10.10.30 -p tcp --dport 5432 -j ACCEPT
        post-down iptables -D FORWARD -s 10.10.10.20 -d 10.10.10.30 -p tcp --dport 5432 -j ACCEPT
        post-up   iptables -A FORWARD -d 10.10.10.30 -p tcp --dport 5432 -j DROP
        post-down iptables -D FORWARD -d 10.10.10.30 -p tcp --dport 5432 -j DROP


source /etc/network/interfaces.d/*
```

Contenu du fichier interfaces de l'hôte (configuration réseau)

Snapshots et sauvegardes

Snapshots des conteneurs LXC

Les snapshots Proxmox permettent de capturer l'état complet d'un conteneur (filesystem + configuration) à un instant donné. Ils exploitent le thin provisioning LVM et ne stockent que les blocs modifiés après le snapshot (copy-on-write), ce qui les rend rapides à créer et peu coûteux en espace disque.

Cette fonctionnalité est essentielle pour sécuriser l'infrastructure avant toute modification risquée (mise à jour, changement de configuration) et pouvoir revenir à un état stable en cas de problème.

Création des snapshots sur les 3 conteneurs (une fois la stack fonctionnelle) :

```
pct snapshot 101 snap-avant-modif --description «État stable avant modifications»
pct snapshot 102 snap-avant-modif --description «État stable avant modifications»
pct snapshot 103 snap-avant-modif --description «État stable avant modifications»
```

Lister les snapshots d'un conteneur :

```
pct listsnapshot <ID>
```

Procédure de restauration (rollback) :

```
pct stop <ID>                          # Arrêter le conteneur avant le rollback
pct rollback <ID> snap-avant-modif     # Restaurer le snapshot
pct start <ID>                         # Redémarrer le conteneur
```

Le rollback supprime toutes les modifications effectuées après le snapshot. Les données ajoutées entre-temps sont perdues.

Un test de rollback a été effectué sur CT2 : suppression volontaire du fichier `docker-compose.yml`, rollback vers `snap-avant-modif`, puis vérification : le fichier est restauré et Kanboard redémarre correctement. Ce test valide que la procédure de restauration est fiable.

Capture d'écran :

```
root@proxmox:~# pct enter 102
root@ct2-kanboard:~# cd /opt/kanboard
root@ct2-kanboard:/opt/kanboard# docker compose down
[+] down 2/2
 ✓ Container kanboard-app   Removed                                                    0.2s
 ✓ Network kanboard_default Removed                                                    0.1s
root@ct2-kanboard:/opt/kanboard# rm docker-compose.yml
root@ct2-kanboard:/opt/kanboard# exit
exit
root@proxmox:~# pct stop 102
root@proxmox:~# pct rollback 102 snap-avant-modif
  Logical volume "vm-102-disk-0" successfully removed.
  WARNING: You have not turned on protection against thin pools running out of space.
  WARNING: Set activation/thin_pool_autoextend_threshold below 100 to trigger automatic extension of thin pools before they get full.
  Logical volume "vm-102-disk-0" created.
  WARNING: Sum of all thin volume sizes (56.00 GiB) exceeds the size of thin pool pve/data and the size of whole volume group (<39.50 GiB).
  Logical volume pve/vm-102-disk-0 changed.
root@proxmox:~# pct start 102
root@proxmox:~# pct enter 102
root@ct2-kanboard:~# cd /opt/kanboard
root@ct2-kanboard:/opt/kanboard# cat docker-compose.yml
services:
  kanboard:
    image: kanboard/kanboard:latest          # Image officielle Kanboard (derniere version stable)
    container_name: kanboard-app             # Nom explicite du conteneur Docker
    restart: unless-stopped                  # Redemarre automatiquement sauf arret manuel
    environment:
      DATABASE_URL: ${DATABASE_URL}          # URL de connexion lue depuis .env
    volumes:
      - kanboard_data:/var/www/app/data        # Donnees applicatives persistantes (config, fichiers)
      - kanboard_plugins:/var/www/app/plugins  # Plugins installes, persistent entre mises a jour
    ports:
      - "8080:80"                            # Kanboard ecoute sur 80 en interne, expose  sur 8080 du CT

volumes:
  kanboard_data:                             # Volume pour les donnees applicatives
  kanboard_plugins:                          # Volume pour les plugins
```

Procédure de rollback d'un snapshot sur un CT LXC

Backups complètes sur disque externe

En complément des snapshots LVM, des sauvegardes complètes (vzdump) ont été mises en place sur un disque physique séparé. Cette mesure constitue une bonne pratique en séparant les données de production des sauvegardes — protection supplémentaire en cas de défaillance du disque principal.

La VM hôte dispose de deux disques :

Disque	Taille	Usage
sda	40 Go	OS Proxmox + thin pool LVM (conteneurs CT1/CT2/CT3)
sdb	150 Go	Vierge à l'installation → configuré comme stockage de sauvegarde

Le disque `sdb` a été partitionné, formaté en ext4, monté sur `/mnt/backup` (persistant via `/etc/fstab`), puis ajouté dans Proxmox (Datacenter → Stockage → Ajouter → Répertoire) sous l'ID `backup-disk` avec comme contenu autorisé les fichiers VZDump.

Un backup complet des 3 conteneurs a ensuite été réalisé :

```
vzdump 101 102 103 --storage backup-disk --mode stop --compress zstd
```

- `--mode stop` : arrête brièvement le conteneur pour garantir la cohérence des données
- `--compress zstd` : compression rapide et efficace

Les fichiers de backup sont stockés dans `/mnt/backup/dump/`. La procédure détaillée de partitionnement et montage est documentée dans le README.

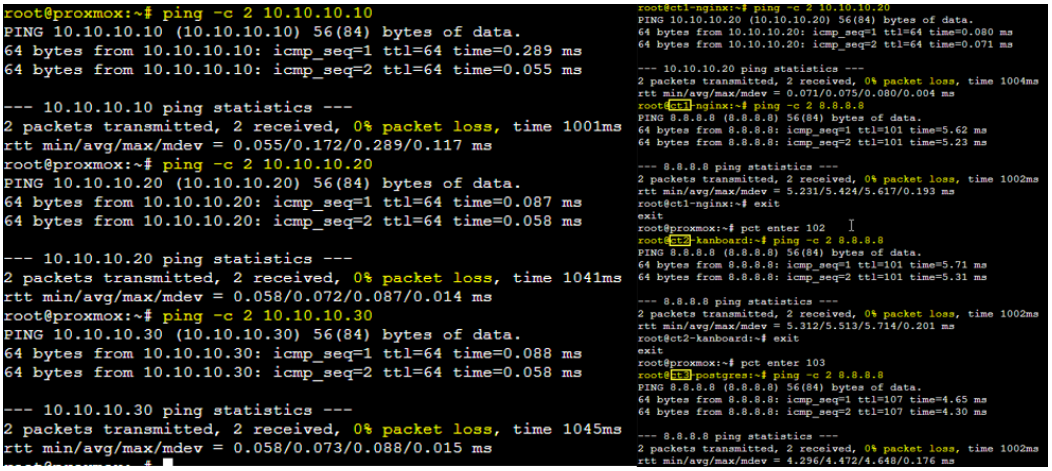
3.2 Description des tests effectués

Les tests suivants ont été réalisés pour valider chaque couche de l'infrastructure, de la connectivité réseau de base jusqu'au fonctionnement bout en bout de la stack applicative.

Connectivité réseau

#	Test	Commande	Résultat attendu	Résultat obtenu
1	Ping host → CT1	ping -c 2 10.10.10.10	0% packet loss	✓ 0% packet loss
2	Ping host → CT2	ping -c 2 10.10.10.20	0% packet loss	✓ 0% packet loss
3	Ping host → CT3	ping -c 2 10.10.10.30	0% packet loss	✓ 0% packet loss
4	Ping CT1 → CT2	ping -c 2 10.10.10.20 (depuis CT1)	0% packet loss	✓ 0% packet loss
5	Accès internet CT	ping -c 2 8.8.8.8 (depuis chaque CT)	0% packet loss	✓ 0% packet loss

Capture d'écran :



Tests de connectivité entre les CT (ping)

Services Docker

#	Test	Commande	Résultat attendu	Résultat obtenu
6	Docker sur CT3	docker compose ps (dans /opt/postgres)	kanboard-db Up	✓ Up
7	Docker sur CT2	docker compose ps (dans /opt/kanboard)	kanboard-app Up (healthy)	✓ Up (healthy)
8	Docker sur CT1	docker compose ps (dans /opt/nginx)	nginx-proxy Up	✓ Up

Capture d'écran :

```
root@ct1-nginx:/opt/nginx# docker compose ps
NAME           IMAGE          COMMAND                  SERVICE   CREATED       STATUS       PORTS
nginx-proxy    nginx:stable   "/docker-entrypoint.…"   nginx     5 hours ago   Up 4 hours   0.0.0.0:80->80/tcp, [::]:80->80/tcp
root@ct1-nginx:/opt/nginx# exit
exit
root@proxmox:~# pct enter 102
root@ct2-kanboard:~# cd /opt/kanboard
root@ct2-kanboard:/opt/kanboard# docker compose ps
NAME            IMAGE                       COMMAND                   SERVICE    CREATED       STATUS                  PORTS
kanboard-app    kanboard/kanboard:latest    "/usr/local/bin/entr…"    kanboard   5 hours ago   Up 4 hours (healthy)    443/tcp, 0.0.0.
0:8080->80/tcp, [::]:8080->80/tcp
root@ct2-kanboard:/opt/kanboard# exit
exit
root@proxmox:~# pct enter 103
root@ct3-postgres:~# cd /opt/postgress
root@ct3-postgres:/opt/postgress# docker compose ps
NAME           IMAGE          COMMAND                   SERVICE    CREATED       STATUS       PORTS
kanboard-db    postgres:16    "docker-entrypoint.s…"    postgres   5 hours ago   Up 4 hours   0.0.0.0:5432->5432/tcp, [::]:5432->5432
/tcp
```

Vérification de l'intégrité des conteneurs sur chaque CT

Bout en bout

#	Test	Commande	Résultat attendu	Résultat obtenu
9	Bout en bout (CT1)	curl -I http://localhost	HTTP 302 → /login	✓ 302 Found
10	Bout en bout (host)	curl -I http://10.10.10.10	HTTP 302 → /login	✓ 302 Found
11	Accès externe (EPFL)	http://10.95.48.44 dans navigateur	Page login Kanboard	✓ Page login affichée
12	Interconnexion DB	Création projet + tâche, vérif psql sur CT3	Données dans PostgreSQL	✓ Projet présent en DB

Capture d'écran t12:

```
id |     name     | is_active | token | last_modified | is_public | is_private |                                                    description                                                    | identi
ault | priority_start | priority_end |         email          | predefined_email_subjects | per_swimlane_task_limits | task_limit | enable_global_tags
----+--------------+----------+-------+--------------+----------+-----------+-----------------------------------------------------------------------------------------------------------------------+-------
-----+----------------+-------------+------------------------+---------------------------+--------------------------+------------+--------------------
 1 | ProjectTest1 | t         |       |    1774341686 | f        | f         | Ceci est un projet de test, cela me permet de vérifier que kanboard communique correctement avec Postgress et qu'il écrit bien dedans. | PT1
 15 |           10 |          25 | antoine.berger@epfl.ch |                           | f                        |         10 | t
(1 row)
```

Requête sur la base de données, preuve de l'ajout correct du contenu par l'app

Sécurité et isolation

#	Test	Commande	Résultat attendu	Résultat obtenu
13	Isolation DB (CT2 → CT3)	psql -h 10.10.10.30 depuis CT2	Connexion réussie	✓ **Connexion OK**
14	Isolation DB (CT1 → CT3)	psql -h 10.10.10.30 depuis CT1	Connexion refusée / timeout	✓ **Timeout**

Capture d'écran :

```
root@ct1-nginx:~# psql -h 10.10.10.30 -U [redacted] -d [redacted] -c "SELECT 1;"
perl: warning: Setting locale failed.
perl: warning: Please check that your locale settings:
        LANGUAGE = (unset),
        LC_ALL = (unset),
        LANG = "en_US.UTF-8"
    are supported and installed on your system.
perl: warning: Falling back to the standard locale ("C").
psql: error: connection to server at "10.10.10.30", port 5432 failed: Connection timed out
        Is the server running on that host and accepting TCP/IP connections?
root@ct1-nginx:~# exit
exit
root@proxmox:~# pct enter 102
root@ct2-kanboard:~# psql -h 10.10.10.30 -U [redacted] -d [redacted] -c "SELECT 1;"
perl: warning: Setting locale failed.
perl: warning: Please check that your locale settings:
        LANGUAGE = (unset),
        LC_ALL = (unset),
        LANG = "en_US.UTF-8"
    are supported and installed on your system.
perl: warning: Falling back to the standard locale ("C").
Password for user [redacted]
 ?column?
----------
        1
(1 row)

root@ct2-kanboard:~#
```

Requête sur la base de données depuis CT1 et CT2 – preuve d'isolation

Persistance et résilience

#	Test	Commande	Résultat attendu	Résultat obtenu
15	Persistance Docker	docker compose down + up (CT3, CT2, CT1)	Données toujours présentes	✓ **Projet et tâche visibles**
16	Snapshot + rollback	Snapshot CT102, suppression fichier, rollback	Fichier restauré, service OK	✓ **Fichier restauré, OK**

3.3 Erreurs restantes

Aucune erreur fonctionnelle n'a été identifiée à la fin du projet. L'ensemble des tests décrits en section 3.2 ont été validés avec succès.

Plusieurs améliorations pourraient cependant être apportées à l'infrastructure :

Monitoring — Un 4ème conteneur LXC pourrait exécuter un service de monitoring tel que Grafana. Cela impliquerait d'adapter les règles iptables pour autoriser CT4 à accéder à la base de données, en utilisant un utilisateur PostgreSQL dédié avec le minimum de privilèges nécessaires.

HTTPS — L'obtention d'un certificat TLS permettrait de passer de HTTP à HTTPS. Cette mesure serait indispensable dans le cadre d'une mise en production, tant pour la sécurité des échanges que pour la crédibilité du service.

Gestion des secrets — Les fichiers `.env` restent exposés sur le disque malgré une restriction d'accès au root. Une solution plus robuste serait l'utilisation de Docker Secrets, qui stocke les identifiants en RAM sans jamais les écrire sur le disque, ou d'un gestionnaire de secrets comme HashiCorp Vault pour une gestion centralisée.

3.4 Liste des documents fournis

Document	Fichier	Description
Rapport TPI blanc	RapportTPI_Blanc-ABerger.docx	Rapport complet du projet (analyse, conception, réalisation, tests, conclusions)
Journal de Travail	JDTBlanc_ABerger.xlsx	Suivi quotidien des activités, problèmes rencontrés et temps effectif
Planification Initiale	PlanificationInitiale_Blanc_ABerger.gantt	Diagramme de Gantt détaillé du projet
Planification finale	PlanificationFinale_Blanc_ABerger.gantt	Diagramme de Gantt détaillé du déroulement réel du projet
Manuel d'installation	README.md	Procédure complète d'installation et fichiers de configuration commentés
Schéma réseau	schema_reseau_TPIBlanc_ABerger.drawio	Schéma d'architecture réseau réalisé avec draw.io

4. Conclusions

Ce projet a été pour moi un vrai défi. N'étant pas spécialisé en Linux, mon expérience jusqu'ici n'avait jamais dépassé les bases théoriques vues en cours durant les modules CFC. Ma dernière utilisation de Docker remontait à deux ans et je n'avais jamais utilisé Proxmox. Malgré cela, je suis satisfait du résultat de mon travail et estime avoir rempli les objectifs initiaux fixés.

#	Objectifs	Statut
1	Installation et configuration de Proxmox VE	**Atteint**
2	Déploiement des conteneurs LXC et communication inter-conteneurs	**Atteint**
3	Stack applicative fonctionnelle	**Atteint**
4	Persistance et résilience	**Atteint**
5	Documentation professionnelle	**Atteint**

Points positifs : en tant que TPI blanc, ce projet m'a permis de découvrir la gestion d'un projet technique et de sa documentation en parallèle. Cette expérience me donne une idée concrète de ce à quoi ressemble un vrai TPI et me met davantage en confiance pour le travail pratique final.

Difficultés rencontrées : les débuts du projet ont été laborieux. Il m'a fallu quelques heures pour me réhabituer à l'environnement Linux, et le blocage initial lié à l'installation de Proxmox (Proxmox non supporté par Ubuntu + KVM non supporté sur la VM xaas) m'a déstabilisé dans mon organisation. Cela m'a cependant appris à gérer une situation critique et à demander de l'aide rapidement à ma cheffe de projet.

Le passage de VMs KVM à des conteneurs LXC a été la principale déviation par rapport au cahier des charges, mais le résultat fonctionnel reste équivalent : trois instances isolées avec Docker, communiquant sur un réseau interne dédié.

Suites possibles : une évolution envisageable serait d'intégrer Entra ID pour lier l'authentification aux bases d'utilisateurs de l'EPFL, rendant l'application accessible aux équipes internes. On pourrait imaginer des espaces collaboratifs par équipe, avec des rôles administrateurs gérés par les supports de proximité ou le ServiceDesk.

5. Annexes

5.1 Résumé du rapport du TPI

Situation de départ

Ce projet consiste à mettre en place une infrastructure virtualisée basée sur Proxmox VE, hébergeant trois conteneurs LXC faisant tourner des services Docker interconnectés : un reverse proxy Nginx, une application de gestion de tâches Kanboard et une base de données PostgreSQL. Le travail a été réalisé sur 3 jours dans le cadre du TPI blanc, sur une VM fournie par le service xaas de l'EPFL. La VM ne supportant pas la virtualisation imbriquée (KVM), des conteneurs LXC ont été utilisés à la place des VMs initialement prévues dans le cahier des charges.

Mise en œuvre

L'installation de Proxmox a d'abord été tentée via les paquets sur Ubuntu 22.04, mais cette approche s'est avérée inadaptée au projet. Une réinstallation en bare-metal depuis l'ISO Proxmox a été réalisée avec l'aide d'un collègue pour l'ajout de l'ISO au catalogue xaas. Un bridge interne (vmbr1) a ensuite été configuré avec NAT pour la communication inter-conteneurs et l'accès internet. Les trois conteneurs LXC ont été créés avec le nesting activé, Docker installé sur chacun, puis les services déployés via docker-compose. Le port forwarding a été mis en place pour rendre Kanboard accessible depuis le réseau EPFL, et des règles iptables ont été ajoutées pour restreindre l'accès à la base de données au seul conteneur applicatif. Des snapshots LXC et des backups sur un disque dédié ont également été configurés.

Résultats

L'ensemble des objectifs du cahier des charges a été atteint. La stack applicative fonctionne de bout en bout : les requêtes HTTP arrivent sur Nginx (CT1), sont redirigées vers Kanboard (CT2), qui communique avec PostgreSQL (CT3). L'isolation de la base de données, la persistance des données et les procédures de snapshot/restauration ont été validées par des tests fonctionnels documentés.

5.2 Sources – Bibliographie

- Documentation officielle Proxmox VE : https://pve.proxmox.com/wiki/Main_Page
- Documentation Docker : https://docs.docker.com/
- Documentation docker-compose : https://docs.docker.com/compose/
- Documentation Kanboard : https://docs.kanboard.org/
- Documentation Nginx : https://nginx.org/en/docs/
- Documentation PostgreSQL : https://www.postgresql.org/docs/
- IA générative Claude : https://claude.ai/
- ISO Installation proxmox : https://www.proxmox.com/en/downloads/proxmox-virtual-environment/iso
- Installation Proxmox sur Ubuntu ? : https://www.reddit.com/r/Proxmox/comments/law3j5/can_i_install_proxmox_ve_on_existing_ubuntu/
- Explication visuelle Reverse Proxy : https://www.youtube.com/watch?v=MpP02aZPSNQ

- Explication visuelle Proxmox : https://www.youtube.com/watch?v=IX5EsbWd8qQ&t

Aides externes :

- Maeva Londero, Cheffe de projet, EPFL – Validation de l'impossibilité de mettre en place le projet sur Ubuntu.
- Nicolas Dubois, EPFL -- Ajout de l'ISO Proxmox VE dans le catalogue xaas (droits d'accès insuffisants pour le faire moi-même)
- IA générative Claude (https://claude.ai/) – Assistance pour Planification + Installation + configuration / infrastructure du projet. Aide à la rédaction du rapport et de la syntaxe générale.

5.3 Journal de travail

Voir JDTBlanc_ABerger.xlsx

5.4 Manuel d'installation

Voir Readme.md

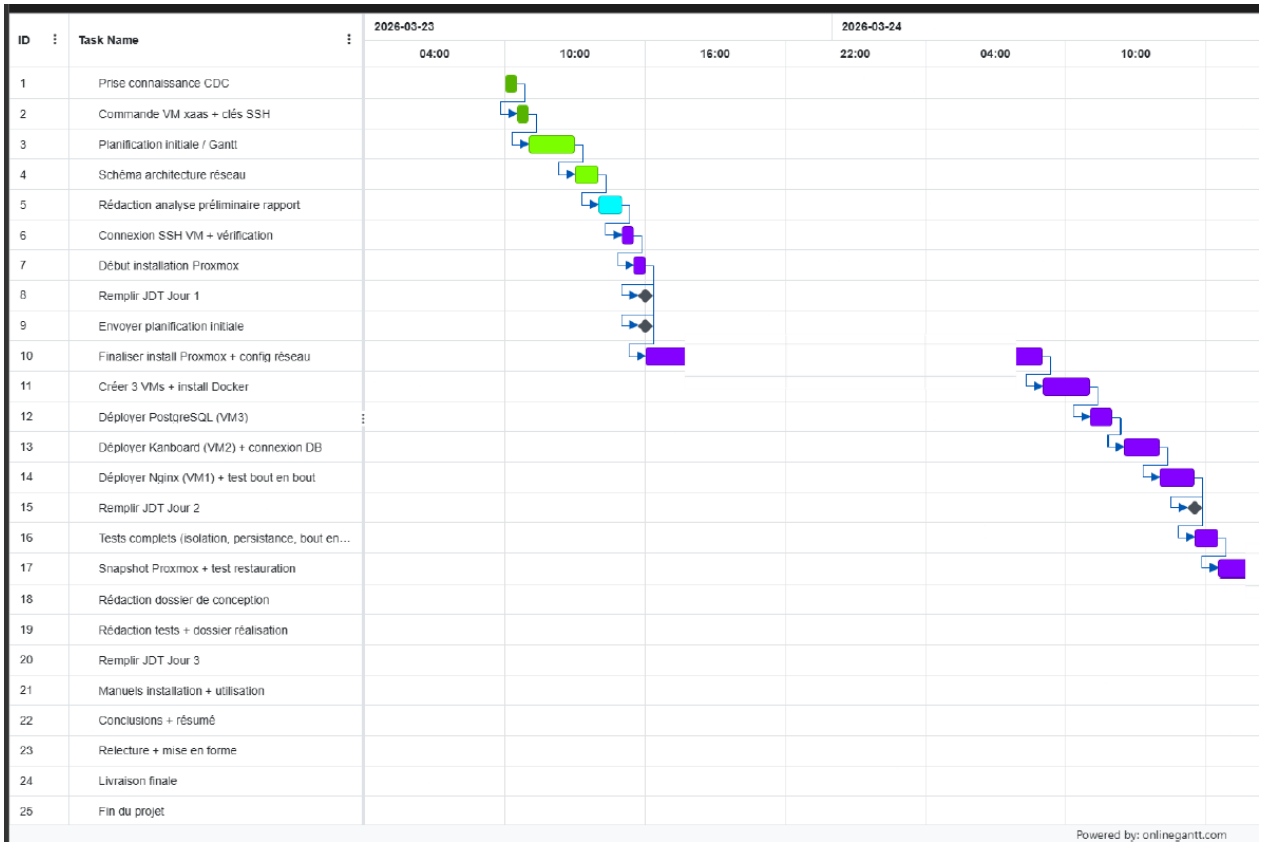
5.5 Glossaire

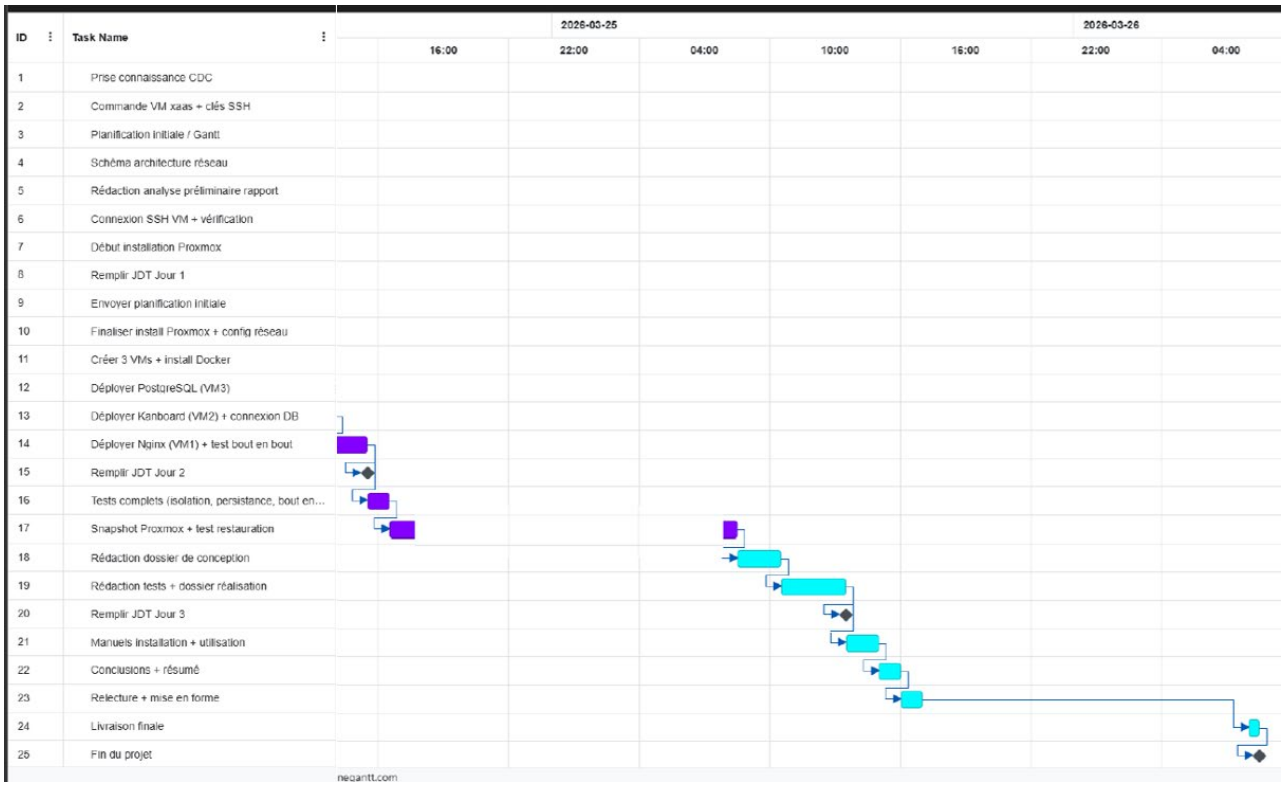
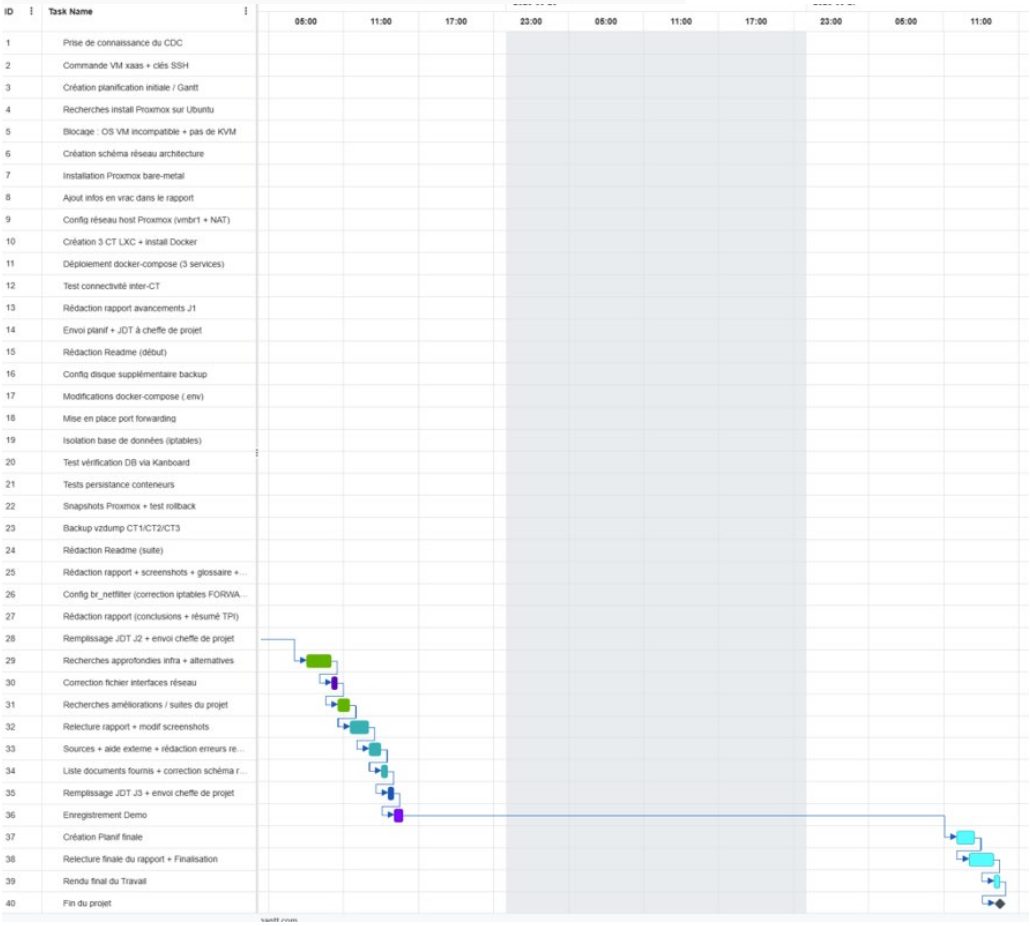
Terme	Définition
Bridge (réseau)	Commutateur virtuel permettant de relier plusieurs interfaces réseau au niveau couche 2 (Ethernet).
CDC	Cahier des charges — document décrivant les exigences et livrables du projet.
CT	Conteneur (Container) — abréviation utilisée pour désigner un conteneur LXC dans Proxmox.
Docker	Plateforme de conteneurisation applicative permettant d'exécuter des applications dans des environnements isolés.
docker-compose	Outil permettant de définir et gérer des applications multi-conteneurs via un fichier YAML.
EPFL	École polytechnique fédérale de Lausanne.
iptables	Outil Linux de filtrage de paquets et de configuration du pare-feu (firewall) au niveau du noyau.
KVM	Kernel-based Virtual Machine — technologie de virtualisation matérielle intégrée au noyau Linux.
LXC	Linux Containers — technologie de conteneurisation au niveau OS, partageant le noyau de l'hôte.
MASQUERADE	Règle iptables de NAT source qui remplace l'IP privée d'un paquet sortant par l'IP publique de l'hôte.
NAT	Network Address Translation — traduction d'adresses réseau permettant à des machines en IP privée d'accéder à internet.
Nesting	Option Proxmox permettant d'exécuter des conteneurs (Docker) à l'intérieur d'un conteneur LXC. (imbriquation)
Proxmox VE	Proxmox Virtual Environment — plateforme open-source de gestion de virtualisation (VMs et conteneurs).
Reverse proxy	Serveur intermédiaire qui reçoit les requêtes clients et les redirige vers le serveur applicatif en amont.
Snapshot	Capture de l'état complet d'un conteneur ou d'une VM à un instant donné, permettant une restauration ultérieure.
vmbr0 / vmbr1	Bridges virtuels Proxmox. vmbr0 = réseau EPFL (externe), vmbr1 = réseau interne privé (10.10.10.0/24).
xaas	Service EPFL fournissant des VMs à la demande (X as a Service).

5.6 Archives du projet

L'ensemble des livrables listés en section 3.4 est remis sous forme d'archive ZIP nommée **TPI_Blanc_AntoineBerger.zip.**
L'archive est disponible à l'adresse :
https://github.com/ACONIIT/TPIBLANC_ABerger_03_2026/

Planification Initiale





Planification Finale